# Q-LEARNING IN TIC-TAC-TOE

*Vom Spielbrett zur lernenden Policy – Schritt für Schritt selbst implementieren*

Informatik · Python · Schüler-Arbeitsheft

**Dein Auftrag** Game.py stellt das Spiel bereit. In QLearningAgent.py sind Vorbereitung und Brett-Hilfsmethoden eingetragen. Du programmierst gezielt die beiden Kernmethoden: ε-greedy-Aktionswahl und Q-Update. Den Episodenablauf überträgst du in drei Codebausteinen aus dem Heft in train_agent und prüfst danach die Lernkurve.

## Spielmodell auf einen Blick

| Rolle | Festlegung |
|---|---|
| Lernender Agent | O; wählt nur aus freien Feldern |
| Gegner | X; beginnt jede Trainingspartie und wählt zufällig |
| Aktion | Feldnummer 0 bis 8; zeile * 3 + spalte |
| Belohnungen | Sieg +1 · Niederlage −1 · Remis 0 |
| Lernverfahren | Q-Learning; nur tatsächlich gespielte Züge aktualisieren die Tabelle |

## Inhaltsverzeichnis


| Kapitel | Was du erarbeitest |
|---|---|
| 1 · Projekt und Umgebung | Spielcode lesen; vorbereitete Startwerte prüfen |
| 2 · Zustände, Aktionen, Folgebretter | Hashbarer State, freie Felder, mögliche Nachfolger |
| 3 · Q-Tabelle | Dictionary-Modell, neun Werte pro Zustand |
| 4 · ε-greedy | Exploration, Exploitation, legale Aktionen, Gleichstände |
| 5 · Q-Update | Terminale Fälle und Bootstrap über legale Folgezuege |
| 6 · Training | Episode als Übergangsfolge gegen zufälliges X |
| 7 · Prüfen und Fehlersuche | Methodentests, Invarianten und typische Bugs |
| 8 · Lernkurve auswerten | Fenster, Testläufe, Schwankungen interpretieren |
| 9 · Grenzen und Transfer | Q-Tabelle, Deep RL, Rewards und Anwendungen |
| 10 · Erweiterungen | Startrolle wechseln und Self-Play |

# 1 · Projekt verstehen, bevor du programmierst

Öffne zuerst Game.py. Die Umgebung verwaltet Brett und Spielregeln; der Agent liest den Zustand, wählt Aktionen und lernt aus dem Spielausgang. Lies den Code, statt Spielregeln doppelt zu programmieren. In QLearningAgent.py sind Hilfsmethoden bereits vorbereitet. choose_action und update_q_table implementierst du selbst; train_agent überträgst du als vollständige Methode aus drei Codebausteinen dieses Hefts. Entferne nach A, B und C jeweils die zugehörige Platzhalterzeile, sonst bricht die Methode trotz ergänztem Code ab.

## Methode 1: __init__(...) – bereitgestellt

Der Konstruktor ist fertig. Lies nach, wie Umgebung, alpha, gamma, epsilon und seine Abnahme sowie die drei Rewards als Attribute gespeichert werden. Die Q-Tabelle beginnt als leeres Dictionary; der eigene Zufallsgenerator macht einen Trainingsseed reproduzierbar.

**Kontrolliere __init__** Zeige auf, welche Zeilen die Hyperparameter, Rewards und Q-Tabelle initialisieren. Warum darf q_table nicht am Anfang jeder Episode neu erstellt werden?

## Lies Game.py gezielt

| Methode / Attribut | Frage beim Lesen | Brauchst du in QLearningAgent? |
|---|---|---|
| game.board | Wie ist das 3×3-Brett gespeichert? | Ja: aktueller Zustand |
| is_valid_move(row,col) | Wie prüft Game ein Feld? | Zum Verständnis; Agent ermittelt freie Felder selbst |
| check_winner(player) | Wie wird Sieg erkannt? | Ja: Episodenende prüfen |
| is_board_full() | Wie wird Remis erkannt? | Ja: Episodenende prüfen |
| set_agent(agent) | Wie wird Agent an Spiel gebunden? | Nur für Partie gegen Menschen |
| play_game() | Welche Reihenfolge haben menschliche und Computerzüge? | Vergleiche mit Trainingsablauf |

## Spielreihenfolge im Projekt

Im Training beginnt X. X setzt zufällig ein X. Wenn das Spiel noch läuft, ist O – der Agent – am Zug. Nach dem O-Zug wird zuerst ein sofortiger Sieg oder ein Remis geprüft. Falls die Partie weitergeht, zieht X zufällig. Danach prüft der Agent erneut auf Niederlage, Remis oder einen nächsten Entscheidungszustand.

**Wichtige Modellierungsentscheidung** Der Q-State ist das Brett genau dann, wenn O entscheiden soll – also nach dem zufälligen X-Zug. Der X-Zug ist Teil der Umgebung. Der Agent aktualisiert nicht aus hypothetischen Zügen, sondern aus dem beobachteten Verlauf.

Darum setzt die Vorübung zu den möglichen Aktionen des lernenden Agenten jeweils ein O in ein freies Feld. X würde man nur einsetzen, wenn man hypothetische Gegnerantworten auf ein bereits gesetztes O untersuchen wollte. Im Training wird davon genau eine zufällige X-Antwort tatsächlich ausgeführt.

## Vorbereitung · API-Notizen

**1.** Notiere, welche Zeichen ein freies Feld, X und O darstellen.

**2.** Finde im Code, wie Game.board zunächst erzeugt wird und wie ein Zug eingetragen wird.

**3.** Schreibe in einem Satz auf, woran du Gewinn und Remis erkennst.

**4.** Erkläre: Warum braucht der Agent keine eigene zweite Implementierung der Gewinnlinien?

# 2 · Zustand, Aktionen und mögliche Folgebretter

## Zustand und Aktion festlegen

Ein Zustand muss die für die nächste Entscheidung relevante Spielsituation eindeutig beschreiben. Hier verwenden wir das vollständige Brett. Eine Aktion ist die Wahl eines Feldes. Damit die neun Q-Werte stets zur gleichen Brettposition passen, nummerieren wir Felder zeilenweise:

| 0 | 1 | 2 |
|---|---|---|
| 3 | 4 | 5 |
| 6 | 7 | 8 |

Beispiel: Aktion 5 entspricht Zeile 1, Spalte 2. Die Umrechnung lautet: zeile = aktion // 3 und spalte = aktion % 3. Die Gegenrichtung lautet aktion = zeile * 3 + spalte.

## Methode 2: get_state(board) – bereitgestellt

Listen lassen sich verändern und deshalb nicht direkt als Dictionary-Schlüssel verwenden. Wandle jede Zeile in ein Tupel um und das ganze Brett in ein Tupel dieser Tupel. So wird derselbe Brettinhalt zu einem stabilen, hashbaren State.

```
Beispielstruktur (kein vollständiges Brett):
board:      Liste von Listen
state:      Tupel von Tupeln
state muss unveränderbar und hashbar sein
```

**Teste und erkläre get_state** Eingabe ist ein 3×3-Board. Zeige, dass die Methode ein Tupel von Tupeln liefert, das Dictionary-Key sein kann. Prüfe: gleiche Bretter ergeben denselben Schlüssel; das Original wird nicht verändert.

## Methode 3: get_valid_actions(board) – bereitgestellt

Gehe alle neun Positionen durch. Für jedes leere Feld fügst du seine Aktionsnummer der Ergebnisliste hinzu. Die Methode darf keine belegten Felder liefern und das Board nicht verändern.

```
Prüfidee:
leeres Board  → [0, 1, 2, 3, 4, 5, 6, 7, 8]
Brett mit Mitte belegt → alle Nummern ausser 4
volles Brett → []
```

**Teste und erkläre get_valid_actions** Führe die drei Prüfideen am Code nach: leer, Mitte belegt, Brett voll. Erkläre, weshalb nur freie Felder in der Aktionsliste stehen dürfen.

### Methode 4: get_next_states(board, player="O") – bereitgestellt

Diese vorbereitete Vorübung erzeugt pro legaler Aktion eine tiefe Brettkopie. Kontrolliere die Anzahl und den Inhalt der Folgebretter. Für mögliche Agentenzüge gilt player="O". Für hypothetische X-Antworten rufst du die Methode auf einem Brett nach einem hypothetischen O-Zug mit player="X" auf. Diese Aufzählung ist nicht der Q-Update-Max-Term; trainiert wird mit einer tatsächlich gezogenen X-Antwort.

# 3 · Die Q-Tabelle als Dictionary

Der Agent soll für jedes Brett und jedes Feld einen Wert speichern. Q(state, action) schätzt, wie gut es langfristig ist, in diesem Zustand dieses Feld zu wählen. Wir speichern für jeden Zustand eine Liste mit neun Zahlen; Index 0 gehört zu Aktion 0 und so weiter.

```
q_table[state] = [Q(s,0), Q(s,1), ..., Q(s,8)]
Neuer State: [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
```

Ein normales Dictionary wirft einen Fehler, wenn ein unbekannter Schlüssel abgefragt wird. Deshalb braucht die Methode get_q_values(state) eine Regel: Existiert der State noch nicht, lege automatisch neun Nullen an und liefere die Liste zurück. Ein solcher State ist nicht zwingend „nie gesehen“: Er kann bereits einmal angelegt worden sein, obwohl noch kein Eintrag aktualisiert wurde.

### Methode 5: get_q_values(state) – bereitgestellt

Die Methode legt bei Bedarf neun Q-Werte mit dem Startwert 0.0 an und gibt die Liste für diesen Zustand zurück. Verfolge im Code den Unterschied zwischen einem unbekannten Zustand und einem bereits gespeicherten Eintrag.

**Prüfpunkte** Der erste Zugriff auf einen State ergibt genau neun Nullen. Ein zweiter Zugriff auf denselben State liefert dieselbe Q-Wert-Liste. Zwei verschiedene Brettzustände bekommen getrennte Einträge.

### Warum hier zunächst kein NumPy?

Für neun Aktionswerte reichen normale Python-Listen. So konzentrierst du dich auf Zustände, Aktionen und die Update-Regel. NumPy wäre bei grossen numerischen Tabellen nützlich, ist für dieses Projekt aber keine Voraussetzung. Ein späterer Vergleich kann zeigen, wie sich q_table[state] als NumPy-Array verhält.

**Begriffe sauber trennen** Der Reward ist die unmittelbare Rückmeldung aus einer Partie. Q ist die gelernte Schätzung des langfristigen Nutzens eines bestimmten Zuges in einem bestimmten Zustand.

### Arbeitsauftrag 2 · Wörterbuch-Auffrischung

Erkläre anhand von q_table: Was ist Schlüssel, was ist Wert? Warum kann der State als Tupel, das Brett selbst aber nicht als Listenstruktur verwendet werden?

# 4 · Eine Aktion mit ε-greedy wählen

Der Agent muss Bekanntes nutzen und zugleich Alternativen erkunden. ε-greedy bedeutet: Mit Wahrscheinlichkeit ε wählt er zufällig aus den legalen Zügen. Mit Wahrscheinlichkeit 1−ε wählt er einen Zug mit maximalem Q-Wert. Bei gleich hohen besten Werten wird zufällig zwischen den besten legalen Zügen gewählt.

**Für unser Projekt** ε startet bei 0.30 und wird nach jeder Episode mit epsilon_decay = 0.9995 multipliziert, aber nie kleiner als epsilon_min = 0.02. Die Werte sind Startparameter zum Experimentieren, keine universell optimalen Konstanten.

## Methode 6: choose_action(board=None) – TODO A

**1.** Wenn kein Brett-Argument übergeben wird, verwende das aktuelle game.board.

**2.** Ermittle die legalen Aktionen. Wenn keine Aktion verfügbar ist, brich mit einer klaren Fehlermeldung ab – eine volle Partie sollte vorher als terminal erkannt werden.

**3.** Hole über get_state und get_q_values die Q-Werte des aktuellen Brettzustands.

**4.** Ziehe eine Zufallszahl aus dem Intervall [0, 1). Vergleiche sie mit epsilon.

**5.** Exploration: Wähle zufällig aus valid_actions, nicht aus allen neun Nummern.

**6.** Exploitation: Berechne den höchsten Q-Wert nur unter den legalen Aktionen. Sammle danach alle legalen Aktionen mit diesem Wert.

**7.** Gib bei Gleichstand eine zufällige der besten Aktionen zurück.

## Warum nur legale Aktionen?

Ein belegtes Feld ist keine verfügbare Handlung. Würdest du das Maximum über alle neun Q-Werte bilden, könnte ein hoher Wert auf einem bereits belegten Feld die Wahl beeinflussen. Deshalb müssen Aktionsauswahl und späterer Zukunftsmaximalwert beide auf die aktuell legalen Aktionen beschränkt sein.

**Manueller Test** Setze epsilon = 0.0 und gib drei legalen Aktionen Werte −0.4, 0.7 und 0.7. Welche Aktionen dürfen zurückkommen? Setze danach epsilon = 1.0. Welche Aktionen dürfen dann zurückkommen?

## Mini-Test nach dem Programmieren

- Die Methode gibt immer eine Zahl von 0 bis 8 zurück.
- Die zurückgegebene Aktion ist immer legal.
- Bei epsilon = 0 wird nur unter besten legalen Q-Aktionen gewählt.
- Bei epsilon = 1 wird zufällig, aber weiterhin nur legal gewählt.

# 5 · Die Q-Update-Regel korrekt umsetzen

Nach einem tatsächlich gespielten Agentenzug und der Reaktion der Umgebung liegt ein Übergang vor: alter Zustand s, gewählte Aktion a, Reward r und – falls das Spiel weitergeht – der nächste Entscheidungszustand s′. Nur Q(s,a) wird angepasst; andere Felder der Tabelle bleiben unverändert.

```
Nicht-terminal:
Target = r + gamma * max(Q(s_next, a_next))
Q(s,a) = Q(s,a) + alpha * (Target - Q(s,a))

Terminal:
Target = r                 # kein Zukunftsterm
```

## Methode 7: update_q_table(...) – TODO B

**1.** Hole die aktuelle Schätzung old_value = Q(state, action).

**2.** Wenn terminal=True: setze target auf reward. next_state darf fehlen; rufe für terminale Zustände keinen legal-action-Maximumwert ab.

**3.** Sonst: Ermittle aus next_state alle legalen Aktionen. Lies für jede Aktion ihren Q-Wert und bilde den höchsten legalen Wert.

**4.** Setze target = reward + gamma * best_next_value.

**5.** Berechne new_value = old_value + alpha * (target − old_value) und speichere es an genau Q(state, action).

**Achtung: Terminal ist eine Information** Ein Brett nach einem Gewinn, einer Niederlage oder einem Remis hat in dieser Episode keine Zukunft. Würdest du dort trotzdem max Q addieren, könnte der Agent terminale Fantasiewerte übernehmen.

## Rechencheck

Angenommen old_value = 0.20, reward = 0, gamma = 0.90 und der höchste legale Folgewert ist 0.60. Dann ist target = 0.54. Bei alpha = 0.20 wird ein Fünftel des Fehlers target − old_value übernommen. Rechne den neuen Q-Wert aus und prüfe: Er muss zwischen 0.20 und 0.54 liegen.

## Welche Folgezustände nutzt der Max-Term?

Die Methode durchsucht nicht alle möglichen Spielverläufe. Sie nimmt genau den tatsächlich beobachteten next_state nach dem realen Agentenzug und realen Zufallszug des Gegners. Innerhalb dieses next_state vergleicht sie die Q-Werte der legalen O-Aktionen.

# 6 · Eine Trainings-Episode als Ablauf

Trainiert wird immer dieselbe Policy-Art: O ist der lernende Agent, X spielt zufällig und beginnt jede Partie. Eine Episode läuft bis Sieg, Niederlage oder Remis. Die Q-Tabelle bleibt über Episoden hinweg erhalten. Der Episodenablauf wird aus den drei Codebausteinen im Heft in das Gerüst übertragen. Nutze die folgende Liste als Lesespur, um die Logik zu erklären und typische Fehler zu erkennen.

## Präziser Ablauf einer Episode

**1.** Setze game.board auf ein neues leeres 3×3-Brett.

**2.** X zieht zuerst zufällig aus allen freien Feldern und setzt ein X.

**3.** Prüfe sofort: Hat X gewonnen? Ist das Brett voll? Wenn ja, Episode endet; es gab noch keine O-Aktion, daher gibt es kein Q-Update.

**4.** Erzeuge vor dem Zug state = get_state(board); so speicherst du einen stabilen Schlüssel, keine veränderbare Board-Referenz.

**5.** Wähle action = choose_action(board). Setze O an dieser Aktion.

**6.** Prüfe sofort den Spielausgang. O-Sieg: reward +1, terminales Update. Brett voll: reward 0, terminales Update.

**7.** Falls das Spiel weitergeht: X wählt zufällig ein freies Feld und setzt X.

**8.** Prüfe erneut. X-Sieg: reward −1 und terminales Update für die vorausgehende O-Aktion. Remis: reward 0 und terminales Update.

**9.** Andernfalls: next_state = get_state(board) nach Xs Antwort. Führe ein nicht-terminales Update mit state, action, reward=0 und next_state aus.

**10.** Speichere am Episodenende genau ein Ergebnis: win, loss oder draw. Reduziere epsilon und begrenze es nach unten durch epsilon_min.

**Übergang – was ist s′?** Für ein nicht-terminales Update ist s′ das Brett nach der Antwort des zufälligen X-Gegners. Nicht das Brett direkt nach O, denn dann wäre X noch am Zug und die nächste Entscheidung läge nicht beim Agenten.

## Methode 8: train_agent(episodes, seed) – aus dem Arbeitsheft übertragen

Die Trainingsmethode verbindet Spielumgebung und Q-Learning. Damit du den langen Ablauf nicht selbst abtippen oder entwerfen musst, setzt du ihn aus drei Codebausteinen zusammen. Jeder Baustein schliesst direkt an den vorherigen an; Einrückungen und Reihenfolge sind Teil des Codes.

**So überträgst du den Code** Suche in QLearningAgent.py die Methode train_agent mit TODO C. Entferne diese ganze Platzhaltermethode. Kopiere danach Block 1, darunter Block 2 und zuletzt Block 3. Die drei Blöcke bilden zusammen genau eine Methode. Übernimm die Einrückungen unverändert und achte darauf, dass TODO A und TODO B vorher ergänzt sind.

## Codebaustein 1 von 3 · Episode vorbereiten und O zieht

Dieser Teil gehört auf Klassenebene und enthält die Methodendefinition. Er setzt den Seed einmal, leert das Brett pro Episode, lässt X eröffnen und speichert vor dem O-Zug den Zustand.

```
    def train_agent(self, episodes=20_000, seed=None):
        # Trainiere gegen zufälliges X und gib ein Ergebnis pro Episode zurück.
        if seed is not None:
            self.rng.seed(seed)

        history = []
        for _ in range(episodes):
            self.game.board = [[" " for _ in range(3)] for _ in range(3)]
            board = self.game.board
            result = None

            # X beginnt jede Partie mit einem zufälligen legalen Zug.
            x_action = self.rng.choice(self.get_valid_actions(board))
            row, col = divmod(x_action, 3)
            board[row][col] = "X"

            while result is None:
                # Zustand und Aktion des Agenten vor dem Zug sichern.
                state = self.get_state(board)
                action = self.choose_action(board)
                row, col = divmod(action, 3)
                board[row][col] = "O"
```

## Codebaustein 2 von 3 · Spielende prüfen und Folgezustand behandeln

Füge diesen Block unmittelbar nach der letzten Zeile von Baustein 1 ein. Er ist weiterhin innerhalb der while-Schleife eingerückt. Terminale Fälle erhalten ein terminales Update; nur wenn das Spiel weitergeht, zieht X zufällig.

```
                if self.game.check_winner("O"):
                    self.update_q_table(
                        state, action, self.reward_win, terminal=True
                    )
                    result = "win"
                elif self.game.is_board_full():
                    self.update_q_table(
                        state, action, self.reward_draw, terminal=True
                    )
                    result = "draw"
                else:
                    # Die Umgebung führt genau eine Zufallsantwort aus.
                    x_action = self.rng.choice(self.get_valid_actions(board))
                    row, col = divmod(x_action, 3)
                    board[row][col] = "X"

                    if self.game.check_winner("X"):
                        self.update_q_table(
                            state, action, self.reward_loss, terminal=True
                        )
                        result = "loss"
                    elif self.game.is_board_full():
                        self.update_q_table(
                            state, action, self.reward_draw, terminal=True
                        )
                        result = "draw"
                    else:
                        # s' ist das Brett nach Xs realer Antwort.
                        next_state = self.get_state(board)
                        self.update_q_table(
                            state,
                            action,
                            reward=0,
                            next_state=next_state,
                            terminal=False,
                        )
```

## Codebaustein 3 von 3 · Ergebnis speichern und epsilon senken

Füge diesen letzten Teil nach Baustein 2 ein, aber wieder ausserhalb der while-Schleife und innerhalb der for-Schleife. return history steht am Schluss ausserhalb der for-Schleife.

```
            history.append(result)
            self.epsilon = max(
                self.epsilon_min,
                self.epsilon * self.epsilon_decay,
            )

        return history
```

Lies die zusammengesetzte Methode einmal von oben nach unten: Ein Episodenergebnis wird genau einmal gespeichert; die Q-Tabelle wird nicht pro Episode gelöscht. Bei terminalen Updates wird kein Zukunftswert verwendet. Bei einer laufenden Partie entsteht next_state erst nach der tatsächlichen X-Antwort.

### Falltabelle für die Entscheidung nach O

| Was passiert? | Reward | Update? | Zukunftsterm? |
|---|---|---|---|
| O gewinnt direkt | +1 | Ja, für den O-Zug | Nein |
| O füllt letztes Feld ohne Sieg | 0 | Ja, für den O-Zug | Nein |
| X gewinnt nach O | −1 | Ja, für den vorausgehenden O-Zug | Nein |
| X füllt letztes Feld | 0 | Ja, für den vorausgehenden O-Zug | Nein |
| Partie läuft nach X weiter | 0 | Ja, für den O-Zug | Ja, max über legale O-Züge in $s'$ |

# 7 · Methoden testen und Fehler finden

Implementiere zuerst TODO A, teste es mit legalen und belegten Feldern, und ergänze dann TODO B. Führe passende Tests aus und behebe Fehler, bevor du fortfährst. Ein kurzer Print-Test ist am Anfang sinnvoll; später helfen Assertions oder ein Testskript.

## Reihenfolge und Akzeptanzkriterien

Die Methoden 1–5 sind vorbereitet und werden getestet. Du implementierst Methode 6 (TODO A) und Methode 7 (TODO B) und überträgst danach Methode 8 (TODO C) aus den drei Codebausteinen.

| Methode | Mindesttest vor dem nächsten Schritt |
|---|---|
| __init__ | Parameter als Attribute; q_table startet leer. |
| get_q_values | Neuer State: 9 Nullen; derselbe State liefert wieder dieselbe Liste. |
| get_state | Gleicher Brettinhalt → gleicher Tupelschlüssel; Schlüssel lässt sich im Dictionary verwenden. |
| get_valid_actions | Leeres Brett liefert 9 Aktionen; ein belegtes Feld wird ausgeschlossen; volles Brett liefert []. |
| get_next_states | Anzahl Folgebretter = Zahl legaler Aktionen; jede Kopie unterscheidet sich an genau einem freien Feld; Original bleibt gleich. |
| choose_action | Exploration und Exploitation liefern ausschliesslich legale Aktionen; Ties werden nicht immer auf Position 0 festgelegt. |
| update_q_table | Nur das Paar (state, action) ändert sich; terminales Target ist gleich Reward; Zukunftsmaximum ignoriert belegte Felder. |
| train_agent | Drei Codebausteine in richtiger Reihenfolge; Seed reproduzierbar; Ergebnislabels win/loss/draw; epsilon sinkt nicht unter Minimum. |

## Typische Fehler – zuerst hier suchen

- Listen als Dictionary-Key verwenden: ein Board als verschachtelte Liste ist nicht hashbar. Erzeuge den Tupel-State.
- board = game.board speichern und danach mutieren: state und next_state verweisen dann auf dasselbe veränderte Objekt. State-Schlüssel sofort erstellen; Brettkopien für hypothetische Nachfolger verwenden.
- Alle neun Aktionen maximieren: belegte Felder müssen ausgeschlossen sein – sowohl bei choose_action als auch beim Bootstrap-Maximum.
- Q-Tabelle bei jedem Episodenstart leeren: das verhindert Lernen über mehrere Partien.
- Terminalzustand bootstrappen: beim Spielende nur den unmittelbaren Reward als Target nehmen.

- Ein Q-Paar ohne Agentenzug aktualisieren: Ein Update braucht immer einen tatsächlich gewählten O-Zug und dessen Zustand.
- Gleichstand mit erstem Index auflösen: frühe Q-Werte sind oft gleich (zunächst alle null); wähle zufällig unter den besten legalen Aktionen.

### Debugging-Protokoll

**1.** Welche Eingabe hatte die Methode?

**2.** Was war die erwartete Ausgabe?

**3.** Welche Zwischengrösse war zuerst falsch: state, valid_actions, reward, next_state oder target?

# 8 · Lernkurve auswerten

Speichere pro Episode ein Ergebnis-Label (win, loss oder draw), nicht nur kumulierte Gesamtzahlen. Die drei Quoten beschreiben zusammen, wie sich die Ergebnisse im Verlauf verändern. Ein einzelner Trainingslauf beweist nicht, dass die Policy gut ist: Zufällige X-Züge und Exploration lassen die Resultate schwanken.

### Ergebnisse in gleich grossen Fenstern zusammenfassen

Teile die Episode-Ergebnisse zum Beispiel in Abschnitte zu je 500 Partien. Berechne für jedes Fenster die Sieg-, Niederlagen- und Remisquote. Das Fensterende ist der Punkt auf der x-Achse; die Quote liegt zwischen 0 und 1 auf der y-Achse.

```
window_size = 500
for start in range(0, len(history), window_size):
    window = history[start:start + window_size]
    n = len(window)  # letztes Fenster kann kleiner sein
    win_rate = window.count("win") / n
    loss_rate = window.count("loss") / n
    draw_rate = window.count("draw") / n
    print(start + 1, start + n, win_rate, loss_rate, draw_rate)
```

**Beim Zeichnen** Zeichne drei Linien: Siegquote, Niederlagenquote und Remisquote. Markiere im Titel die Fenstergrösse und ob die Daten aus dem Training oder aus einem separaten Testlauf stammen.

### Eine Kurve ist kein Beweis

| Beobachtung | Mögliche Deutung und nächste Prüfung |
|---|---|
| Siegquote steigt, Niederlagenquote sinkt | Der Agent wird gegen genau diesen Zufallsgegner besser. Prüfe zusätzlich Remisquote und Testpartien. |
| Remisquote steigt | Kann Fortschritt sein, wenn Niederlagen zurückgehen. Gegen einen Zufallsgegner ist eine hohe Remisquote allein aber kein Beweis für optimale Spielweise. |
| Kurve schwankt stark | Zufallsgegner und Exploration erzeugen Varianz. Grössere Fenster und mehrere Trainings-Seeds vergleichen. |
| Kurve bleibt flach | Prüfe zuerst Code, Rewards und legale Aktionen; erst dann alpha, gamma und epsilon verändern. |

### Training von Policy-Test trennen

Während des Trainings ist epsilon oft grösser als null. Ein Teil der Züge wird absichtlich zufällig gewählt. Um die gelernte greedy Policy zu testen, setze epsilon nur für separate Testpartien auf 0 und schalte Q-Updates aus. Teste gegen mehrere feste Gegner oder Seeds. Danach den Trainingswert von epsilon wiederherstellen, falls weitertrainiert wird. Eine gute Kurve gegen den Zufallsgegner zeigt noch keine optimale Tic-Tac-Toe-Strategie.

### Interpretationsfragen

**1.** Was passiert mit Sieg-, Niederlagen- und Remisquote? Beurteile die drei gemeinsam.

**2.** Warum kann die Trainingskurve trotz sinkendem epsilon zunächst schwanken?

**3.** Warum braucht man mehrere Seeds und einen getrennten Policy-Test?

## 9 · Grenzen, Reward-Probleme und Anwendungen

Das Tic-Tac-Toe-Projekt zeigt Q-Learning in einer kleinen, vollständig beobachtbaren und diskreten Umgebung. Die Grundidee lässt sich übertragen. Die konkrete Q-Tabelle stösst jedoch schnell an Grenzen.

### Grenze 1: Die Q-Tabelle wächst mit den Zuständen

Ein 3×3-Brett hat höchstens $3^9 = 19\,683$ mögliche Zellbelegungen, bevor unmögliche oder unerreichbare Bretter ausgeschlossen werden. Für jede Belegung gibt es bis zu neun Aktionswerte. Das ist hier noch handhabbar. Bei grossen Brettspielen, kontinuierlichen Messwerten oder Kamerabildern wird eine Tabelle viel zu gross. Zudem lernt die Tabelle jeden Zustand getrennt: Ähnliche Bretter teilen ihr Wissen nicht automatisch.

### Deep RL: Werte approximieren statt speichern

Deep Q-Learning ersetzt die Tabelle durch ein neuronales Netz. Als Eingabe dient eine codierte Beobachtung, zum Beispiel ein Spielbrett oder ein Bild; als Ausgabe schätzt das Netz die Q-Werte möglicher Aktionen. Das Netz kann Muster über ähnliche Situationen hinweg verallgemeinern. So wurde ein Deep Q-Network etwa auf Atari-Spielen mit Bildpixeln als Eingabe untersucht [2]. Für Tic-Tac-Toe wäre ein Netz unnötig kompliziert: Die Tabelle ist leichter zu verstehen und zu prüfen. Deep RL bringt zusätzliche Daten-, Rechen- und Stabilitätsfragen mit sich.

### Sparse Rewards: Rückmeldung erst am Ende

Unser Agent erhält im Wesentlichen erst beim Partieende $+1$, $-1$ oder 0. Dazwischen gibt es keinen direkten Reward für einen guten Zug. Das ist ein sparsames Belohnungssignal: Ein erfolgreicher Spielausgang muss über mehrere Updates auf frühere Entscheidungen zurückwirken. Bei Tic-Tac-Toe sind Partien kurz. In Aufgaben mit langen Wegen bis zum Ziel kann es sehr schwer sein, überhaupt hilfreiche Rückmeldungen zu erhalten.

### Reward Design und Reward Hacking

Der Agent maximiert die Zahl, die wir als Reward festlegen – nicht automatisch unsere unausgesprochene Absicht. Eine Zusatzbelohnung für jeden Zug in der Mitte könnte beispielsweise die Mitte überbewerten, selbst wenn ein anderer Zug den Sieg sichert. Das ist ein vereinfachtes Reward-Hacking-Risiko: Der Agent optimiert den messbaren Ersatz, nicht zwingend das eigentliche Ziel. Ein reales Anschauungsbeispiel: In einem Rennspiel sammelte ein Agent wiederholt Bonusobjekte in einer Schleife, statt das Rennen zu beenden, weil die Bonuspunkte stärker belohnt wurden [3].

**Gute Kontrollfrage** Wenn der Agent den Reward maximal macht: Würde ich dieses Verhalten tatsächlich als Erfolg bezeichnen? Prüfe auch Zusatzbelohnungen, Strafen und Abbruchregeln.

## Wo wird Reinforcement Learning eingesetzt?

| Bereich | Mögliche Agentenentscheidung | Beispiel für ein Zielsignal |
|---|---|---|
| Spiele | Welcher Zug oder welche Steuerung? | Spielgewinn oder Punktestand |
| Robotik | Wie bewegen sich Motoren? | Aufgabe erledigt, Zeit oder Energie |
| Verkehr / Energie | Wie werden Ampeln oder Speicher gesteuert? | Wartezeit, Sicherheit oder Verbrauch |
| Sprachmodelle mit RLHF | Welche Antwort wird erzeugt? | Menschliche Präferenz, durch ein Reward-Modell geschätzt [4] |

## RLHF: menschliche Präferenzen als Reward

RLHF steht für Reinforcement Learning from Human Feedback. Vereinfacht: Menschen schreiben beispielhafte Antworten und vergleichen alternative Modellantworten. Aus diesen Präferenzen wird ein Reward-Modell trainiert. Anschliessend wird das Sprachmodell so weitertrainiert, dass es höhere vorhergesagte Präferenz-Rewards erzielt [4]. Das ist kein Q-Learning mit einer Tic-Tac-Toe-Q-Tabelle. Auch ein Reward-Modell bleibt ein unvollkommener Stellvertreter für menschliche Absichten.

## Transferfragen

1. Welche Information müsste ein Roboterzustand enthalten, damit eine Aktion sinnvoll gewählt werden kann?
2. Warum ist ein Reward für „schnell fertig werden“ nicht immer gleichbedeutend mit „sicher fertig werden“?
3. Welche gewünschte Absicht könnte ein Reward-Modell bei Sprachantworten verfehlen?

## Quellen zum Vertiefen

[1] Sutton & Barto, Reinforcement Learning: An Introduction, 2. Auflage: http://incompleteideas.net/book/the-book-2nd.html

[2] Mnih et al., Human-level control through deep reinforcement learning, Nature (2015): https://www.nature.com/articles/nature14236

[3] Krakovna et al., Specification gaming: the flip side of AI ingenuity, DeepMind (2020): https://deepmind.google/blog/specification-gaming-the-flip-side-of-ai-ingenuity/

[4] Ouyang et al., Training language models to follow instructions with human feedback (2022): https://arxiv.org/abs/2203.02155

# 10 · Erweiterungen und Transfer

Wenn die Grundversion zuverlässig läuft, kann man das Modell anspruchsvoller machen. Diese Erweiterungen gehören nicht in die erste Implementierung – sie verändern Annahmen, Zustände oder Trainingsdynamik.

## A · Startrolle wechseln

In manchen Episoden beginnt der Agent, in anderen X. Dafür muss der Zustand zusätzlich erkennen lassen, welcher Spieler am Zug ist – ausser die State-Definition garantiert bereits eine eindeutige Perspektive. Die Aktionen, Rewards und Gewinnprüfungen müssen zur wechselnden Markierung passen. Auswertung getrennt nach Startrolle durchführen.

| Neue Frage | Warum wichtig? |
|---|---|
| Wie wird gespeichert, wer am Zug ist? | Gleiches Brett kann aus Sicht anderer Rolle andere Bedeutung haben. |
| Wie heisst „Sieg“ für den Agenten? | Reward muss immer aus Agentensicht definiert sein. |
| Wie werden Startrollen ausgeglichen? | Sonst lernt der Agent überwiegend eine Position. |

## B · Self-Play

Zwei lernende Agenten spielen gegeneinander. Das Gegenüber verändert sich während des Trainings mit – die Datenverteilung ist dadurch nicht stabil. Eine Policy kann gegen den aktuellen Lernpartner besser werden, ohne gegen frühere Versionen oder andere Strategien robust zu sein. Rollen und Markierungen müssen konsistent abgebildet werden; Auswertung gegen feste Gegner bleibt nötig.

## Weitere kleine Erweiterungen

- Agent gegen feste Baseline testen (z. B. zufällig oder einfache Gewinn-/Block-Regeln).
- Q-Tabelle speichern/laden und prüfen, ob die Datenstruktur danach noch dieselbe ist.
- Spiegelungen und Rotationen des Bretts untersuchen: Sie sind strategisch symmetrisch, aber als rohe Tupel verschiedene Schlüssel.
- Belohnungs- und Hyperparameterexperimente mit dokumentierten Seeds durchführen.

**Abschlussprodukt** Du kannst Game.py erklären, die Agentenmethoden selbst implementieren, einen Q-Update von Hand prüfen und Resultate kritisch auswerten. Als nächsten Schritt kann das Notebook Training und Lernkurve begleiten – es ist bewusst nicht Teil dieses Skripts.

## Deine Rückschau

Die Methode, die ich am besten erklären kann, ist …

_____

Ein Bug, den ich selbst gefunden und behoben habe, war …

_____

Eine Annahme unseres Modells ist … _____

Als Nächstes würde ich testen … _____